Computer Science Department

Master Degree in Computer Science

PROJECT DOCUMENTATION for BIG DATA

Link Prediction in Social Networks

Author:

Mattia Curri

ACADEMIC YEAR **2025-2026**

Contents

1 Business Understanding **1**
1.1 Determine Business Objectives . . . 1
1.2 Assess Situation . . . 2
1.3 Determine Data Mining Goals . . . 3

2 Data Understanding **4**
2.1 Collect Initial Data . . . 4
2.2 Describe Data . . . 4
2.3 Explore Data . . . 5
2.4 Verify Data Quality . . . 11
2.5 Summary . . . 13

3 Data Preparation **15**
3.1 Select Data . . . 15
3.2 Clean Data . . . 15
3.3 Construct Data . . . 15
3.4 Integrate Data . . . 15
3.5 Format Data . . . 15

4 Modeling **16**
4.1 Select Modeling Techniques . . . 16
4.2 Generate Test Design . . . 18
Metrics . . . 18
Experiments . . . 18
Examples of network metrics . . . 18
Description of metrics . . . 18
4.3 Build Model . . . 19
Standard Implementation . . . 19
Incremental Implementation . . . 19
4.4 Assess Model . . . 20

5 Evaluation **21**
5.1 Evaluate Results . . . 21
5.2 Review Process . . . 21

6 Deployment **22**

7 Conclusion **23**

1 Business Understanding

This chapter covers the business understanding phase of the CRISP-DM methodology.

1.1 Determine Business Objectives

Background

Social network analysis (SNA) has emerged as a critical pursuit in understanding the complex dynamics of modern online communities. While traditional statistical techniques remain valuable, SNA introduces specialized measures to capture the dependencies between social entities, characterize their behaviors, and assess their impact on the network both structurally and temporally [1]. The fundamental principle of SNA lies in shifting the analytical focus from individual social actors to the relationships and interactions that bind them, revealing patterns that emerge from the network's underlying structure rather than from isolated entities [1].

In the globalized digital landscape, social networks have become one of the favorite means for modern society to perform social interactions and exchange information via the Internet, allowing people to share their contents without boundaries [2, 3]. Through these platforms, individuals communicate and express their comments, likes, and interests in a fast and easy manner, creating complex connections that can be immediately visualized as large graphs [2]. This massive spread of social media has introduced unprecedented opportunities for information dissemination [4]. However, this very interconnectedness presents significant challenges: the same platforms that facilitate legitimate social connections also provide powerful tools for malicious activities, including misinformation campaigns, cyberbullying, hate speech propagation, and the radicalization of vulnerable individuals [5, 3, 4, 6].

Risky users exploit the spreading power of social networks for illegal or harmful activities such as inciting religious or political extremism, disseminating disinformation, or recruiting. For these reasons, detecting them has become a fundamental task for ensuring the safe use of these platforms [3, 5, 6]. This challenge is particularly acute when dealing with borderline users who appear risky from certain perspectives but not from others, and is further complicated by the dynamic nature of social networks [3, 5]. User relationships, content semantics, and behavioral patterns continuously evolve; models that treat networks as static entities fail to capture users who transition from safe to radicalized behavior over time [5].

The unstructured nature of user interactions on social media platforms renders manual identification of malicious actors both tedious and impractical at scale [4]. Recent research has increasingly focused on leveraging graph-based representations, where users constitute nodes and their interactions form edges, to address this complexity [4, 7]. Link prediction, defined as the process of predicting future connections between pairs of unconnected links based on existing connections [2], represents a fundamental problem in network analysis that has evolved from static snapshot-based approaches to temporal methodologies that exploit multiple network states across time to predict future or re-occurring

connections [1, 2]. This temporal perspective aligns with the recognition that real-world networks are not static but dynamic and continuously evolving, requiring analytical frameworks that can capture growth patterns and temporal dependencies [1].

Given the urgency of protecting society from exposure to discriminatory, hateful, and violent content at the massive scale of contemporary social networks, there is a critical need for effective automated systems to assist Law Enforcement Agencies in identifying and addressing users who disseminate malicious content [6]. This work addresses this need by developing methods that integrate multiple perspectives of user behavior, account for the temporal evolution of networks, and leverage the relational structure inherent in social media platforms to enable accurate and timely identification of risky users.

Business Goals

The primary business goal of this project is to develop an effective temporal link prediction system for social networks that can:

- Predict future connections between users based on their historical interactions and content
- Evaluate the effectiveness of incremental learning approaches for temporal network evolution
- Given a dataset of users and relative posts, generate realistic synthetic social network structures
- Provide insights into how network properties evolve over time

Business Success Criteria

The project will be considered successful if it achieves the following outcomes:

- Development of a temporal link prediction model based on EvolveGCN
- Quantitative comparison of model performance using standard metrics (Mean Average Precision, AUC, Precision, Recall, F1)
- Successful transfer learning application: trained models can generate plausible network structures on synthetic datasets
- Analysis of network evolution patterns and structural properties across temporal snapshots
- Generation of a large synthetic network of 1000 artificial users with realistic connection patterns

1.2 Assess Situation

Inventory of Resources

The available data are being collected from the social network platform Gab[1], known for its minimal content moderation policies and popularity among far-right communities, over a defined time period.

Current resources available for the project are the following machines:

1. CPU: AMD Ryzen 7 7700, 8 cores, 16 threads; GPU: NVIDIA RTX 5060Ti, 16 GB VRAM; RAM: 48 GB
2. CPU: AMD Ryzen 7 7840HS, 8 cores, 16 threads; GPU: Integrated AMD Radeon 780M; RAM: 48 GB

[1] https://gab.com/

3. CPU: Intel Core i7-10750H, 6 cores, 12 threads; GPU: NVIDIA GTX 1650 Mobile Max-Q, 4 GB VRAM; RAM: 16 GB

Identify Constraints

There are no significant constraints anticipated for this project. However, potential challenges include:

- Data Quality: The collected data may contain noise, missing values, or inconsistencies that could affect model performance.
- Computational Resources: Training complex models on large datasets may require significant computational power and time.
- Ethical Considerations: Handling user data requires adherence to privacy regulations and ethical guidelines to ensure responsible use of information.

Assumptions

- The dataset from Gab is representative of broader social network dynamics and user behaviors.
- A connection between users exists if they are connected in one snapshot
- Graphs are undirected and unweighted
- If a connection exists between two users in a snapshot, it will also exist in all subsequent snapshots

1.3 Determine Data Mining Goals

Data Mining Goals

Here we translate the business goals into specific data mining objectives:

- Develop a temporal link prediction model using EvolveGCN to forecast future user connections based on historical interaction data.
- Evaluate the model's performance using metrics such as Mean Average Precision (MAP), Area Under the Curve (AUC), Precision, Recall, and F1-score.
- Adapt and test the trained model for generating synthetic social network structures.
- Analyze the temporal evolution of network properties, including degree distribution, clustering coefficient, and average path length.

Success Criteria

The success of the data mining efforts will be measured by:

- Achieving high performance on link prediction tasks
- Demonstrating the model's ability to generalize to synthetic datasets, producing realistic network structures.
- Providing comprehensive insights into the temporal dynamics of the social network through detailed analysis of evolving network metrics.

2 📖 Data Understanding

This chapter covers the data understanding phase of the CRISP-DM methodology.

2.1 Collect Initial Data

The available data are being collected from the social network platform Gab[1]. The dataset includes several snapshots, in particular, 2016-2021, 2022, 2023, 2024, January-July 2025 and July 2025. For every snapshot, there three files:

- posts_current_snapshot.csv: contains all the posts made by users in the platform for that snapshot;
- posts_incremental.csv: contains all the posts made by users in the platform since the previous snapshots up to the current one;
- social_network.edg: contains the social network graph in edge list format, where every line represents a directed edge from a follower to a followee (X follows Y).

Along with them, there are also 3 synthetic batches for the subsequent task of using the trained model on the original data to generate links.

For both the datasets, the embedding has been provided, computed using a BERT model.

2.2 Describe Data

Raw data files are in CSV format, with the exception of the social network graph which is in edge list format. For each post, the following attributes are available:

- unnamed_0 (autoincrement): index of the post, autoincremented
- id (integer): unique identifier of the post
- content (string): text of the post
- account_id (integer): unique identifier of the user who made the post
- created_at (string): ISO 8601 timestamp of when the post was created
- language (string): language code of the post
- replies_count (integer): number of replies to the post
- reblogs_count (integer): number of reblogs of the post

[1] https://gab.com/

- reactions_counts (string): JSON dictionary of reactions to the post
- quotes_count (integer): number of quotes of the post
- timestamp (string): ISO 8601 timestamp of the post (created_at + timezone)

2.3 Explore Data

The exploration of the dataset involves analyzing temporal patterns, content characteristics, user engagement, and activity distributions across all snapshots.

Temporal Distribution

The dataset spans from August 22, 2016 to August 2, 2025, covering a period of approximately 9 years. Table 2.1 summarizes the temporal distribution across all snapshots.

Table 2.1: Temporal distribution of posts across snapshots

Snapshot	Posts	Time Span (days)	Max Gap (days)
2016-2021	18,926	1,957	49
2022	13,108	364	<1
2023	13,376	364	<1
2024	13,400	365	<1
Jan-Jul 25	44,878	180	<1
Jul 25	78,857	32	<1

The early snapshot (2016-2021) shows significant temporal gaps, with 39 gaps of 7 or more days detected, reflecting the sparse data collection in the platform's early years. From 2022 onwards, the data collection became more consistent, with no gaps exceeding one day. The volume of posts shows a marked increase over time, with the Jul 25 snapshot alone containing more posts than all previous years combined.

Figure 2.1 visualizes the temporal distribution of posts across all snapshots, showing a clear growth trend with an exponential increase in 2025.

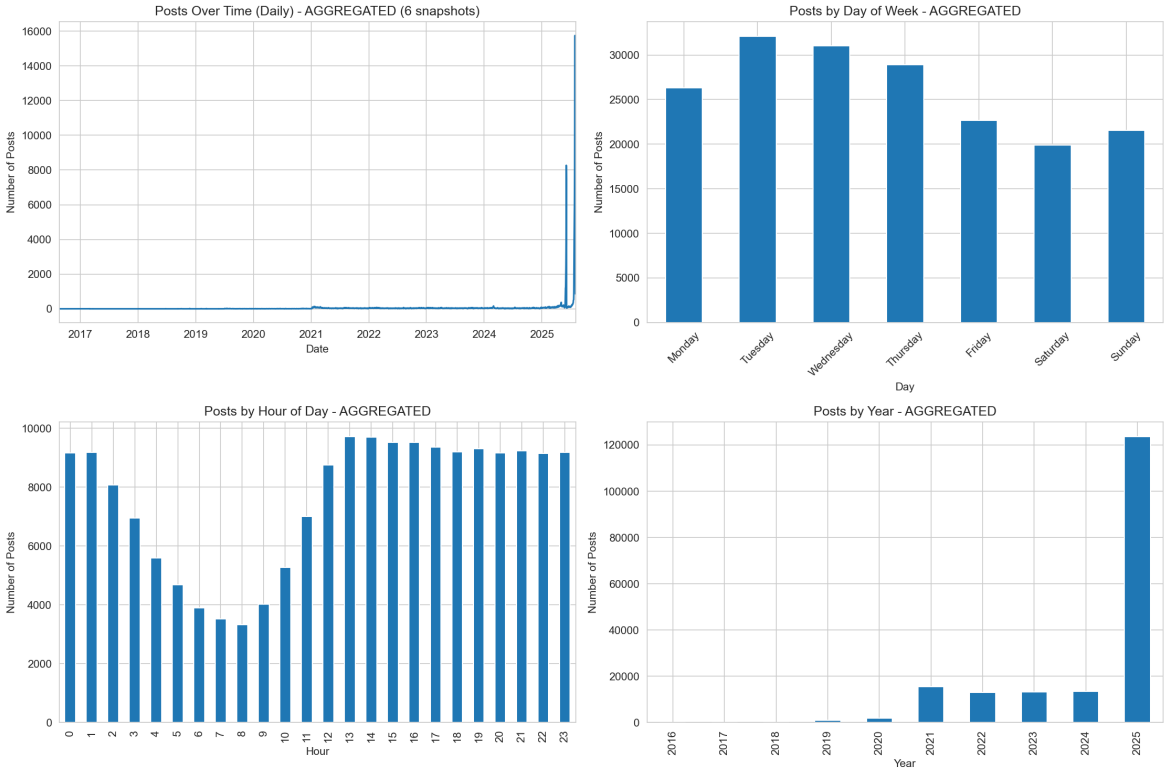

Figure 2.1: Temporal distribution of posts across the aggregated dataset (2016-2025). The histogram shows monthly post counts with a clear exponential growth pattern, particularly evident in 2025.

Language Distribution

The dataset is predominantly English, with 97.01% of posts (177,085 out of 182,545) written in English. A total of 80 unique languages were detected across all snapshots. Table 2.2 shows the top languages.

Table 2.2: Top 10 languages in the aggregated dataset

Language	Posts	Percentage
English (en)	177,085	97.01%
French (fr)	994	0.54%
Unknown	877	0.48%
German (de)	823	0.45%
Spanish (es)	429	0.24%
Japanese (ja)	348	0.19%
Italian (it)	252	0.14%
Portuguese (pt)	230	0.13%
Dutch (nl)	202	0.11%
Polish (pl)	125	0.07%

The English dominance increased over time, from 92.18% in 2016-2021 to 98.70% in Jan-Jul 25, suggesting the platform's user base became increasingly anglophone.

Figure 2.2 shows the language distribution as a pie chart, highlighting the overwhelming dominance of English content.

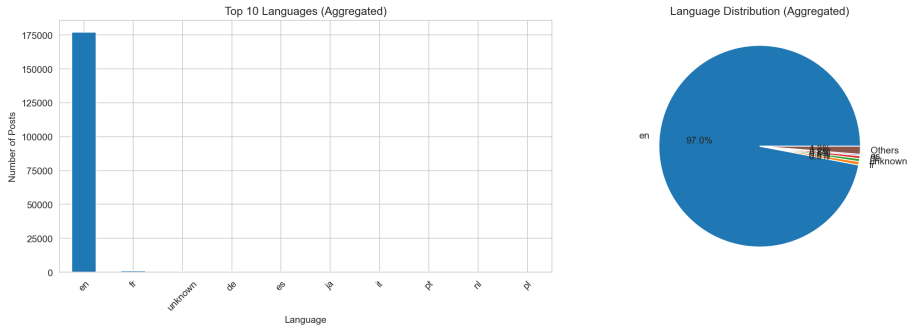

Figure 2.2: Language distribution in the aggregated dataset. English dominates with 97% of all posts, followed by French (0.5%), unknown languages (0.5%), and German (0.5%).

Content Characteristics

Text Length Analysis

Post content varies significantly in length. Table 2.3 summarizes the text length statistics.

Table 2.3: Text length statistics (characters) across the aggregated dataset

Statistic	Value
Mean	247.38
Std. Deviation	552.45
Minimum	1
25th Percentile	37
Median	89
75th Percentile	213
Maximum	11,753

- No empty posts were found in the dataset;
- Very short posts (1-9 characters) account for approximately 4-5% of posts per snapshot;
- Very long posts (>1,000 characters) represent a small but consistent portion, with 9,456 posts total.

The average word count is 35.4 words per post, with a median of 13 words, indicating a right-skewed distribution typical of social media platforms.

Figure 2.3 illustrates the text length distribution, showing the characteristic long-tail pattern.

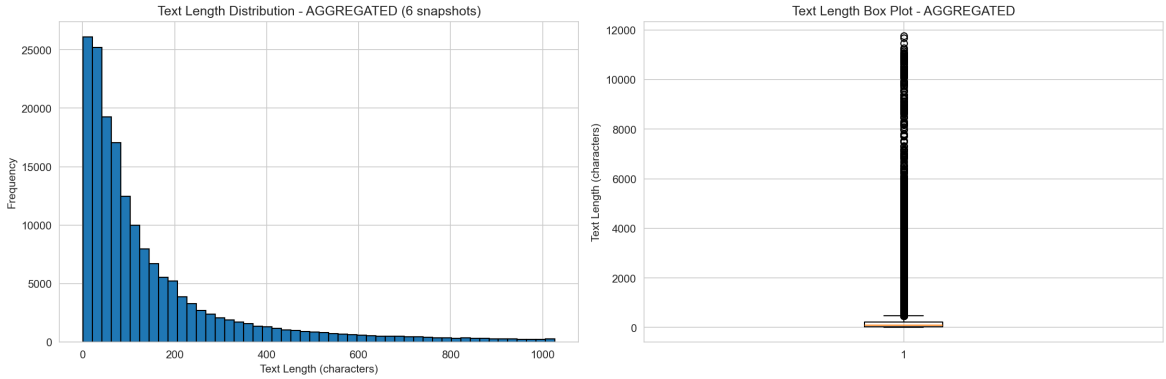

Figure 2.3: Text length distribution (aggregated). The histogram shows character count distribution with a log-scale y-axis, revealing a heavy right-skewed distribution where most posts are short (under 500 characters) but some extend to over 10,000 characters.

URL, Hashtag, and Mention Analysis

Interestingly, the content shows minimal use of URLs, hashtags, and mentions:

- Posts containing URLs: 7 (<0.01%)
- Posts containing hashtags: 0 (0.00%)
- Posts containing mentions: 0 (0.00%)

This suggests that the content has been preprocessed to remove or normalize such elements, or reflects a specific subset of posts that does not include these features.

Engagement Metrics

User engagement is measured through replies, reblogs, and quotes. Table 2.4 presents the engagement statistics for the aggregated dataset.

Table 2.4: Engagement statistics for the aggregated dataset

Metric	Mean	Median	Max
Replies	8.47	0	22,211
Reblogs	28.94	1	24,464
Quotes	1.56	0	2,292
Total Engagement	38.98	3	34,656

A significant portion of posts (28.06%) have zero engagement, while a small number of viral posts drive the high averages. Engagement metrics show a declining trend over time: the 2016-2021 snapshot shows an average total engagement of 156 per post, while the Jul 25 snapshot shows only 7.78, indicating either platform changes or sampling methodology differences.

Figure 2.4 visualizes the engagement distribution, highlighting the heavy-tailed nature of social media interactions.

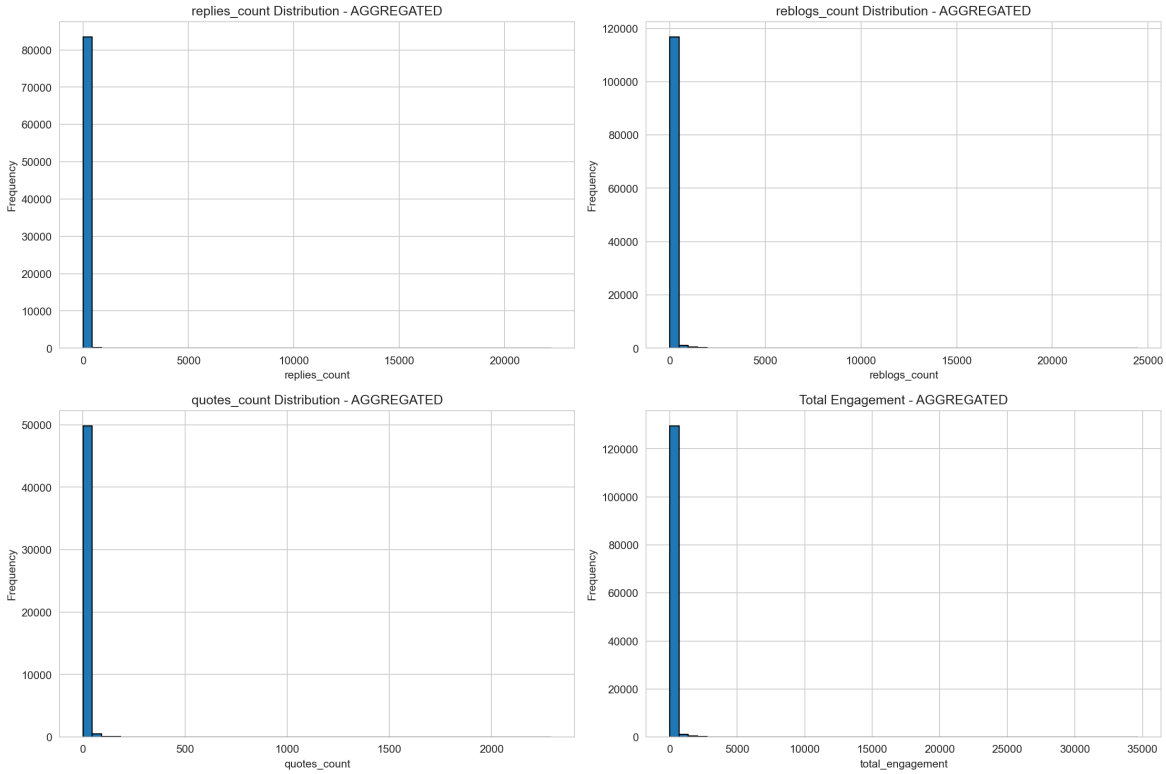

Figure 2.4: Engagement metrics distribution (aggregated). Box plots show replies, reblogs, and quotes counts on a log scale, revealing extreme outliers and the highly skewed nature of engagement metrics.

Reactions Analysis

Reactions provide insight into user sentiment. The platform supports multiple reaction types, with reaction type "1" (likely a "like" equivalent) being dominant, accounting for over 90% of all reactions. Posts with no reactions represent 23.11% of the dataset. The average number of reactions per post is 62.22, with a highly skewed distribution (median of 3).

Figure 2.5 shows the distribution of reaction types across the aggregated dataset.

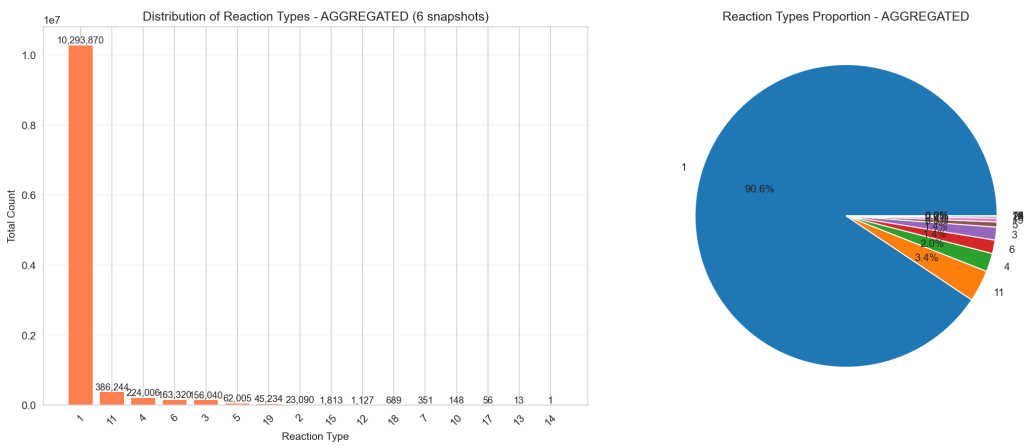

Figure 2.5: Reaction types distribution (aggregated). The bar chart shows counts per reaction type on a log scale, with type "1" (standard like) dominating overwhelmingly, followed by types "11", "4", and "6".

User Activity Patterns

The dataset contains posts from 27,002 unique users, with an average of 6.76 posts per user. Table 2.5 categorizes users by activity level.

Table 2.5: User activity categories

Category	Users	Percentage
Single post users	10,179	37.70%
Casual users (2-10 posts)	12,558	46.51%
Active users (11-100 posts)	4,149	15.37%
Power users (>100 posts)	116	0.43%

The top 1% of users contribute disproportionately to the content: the top 10 most active users generated a combined 4,394 posts. The most active user (account_id 3652359) authored 743 posts across all snapshots.

Figure 2.6 illustrates the user activity distribution, showing the typical power-law pattern where few users contribute most content.

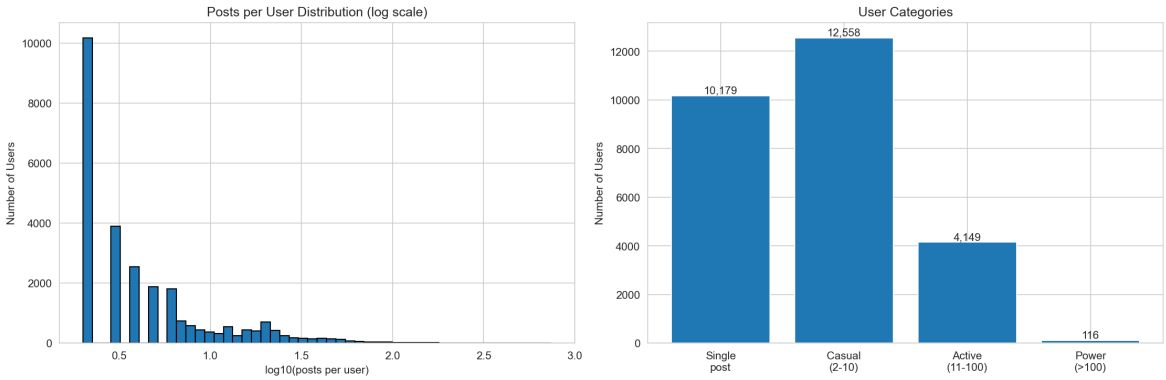

Figure 2.6: User activity distribution (aggregated). The cumulative distribution shows that approximately 85% of users posted 10 or fewer times, while a small fraction of power users account for a disproportionate share of total posts.

Content Analysis: Word Clouds and N-grams

To better understand the topics discussed on the platform, word frequency analysis was performed. Figure 2.7 presents a word cloud visualization of the most frequent terms in the aggregated dataset.

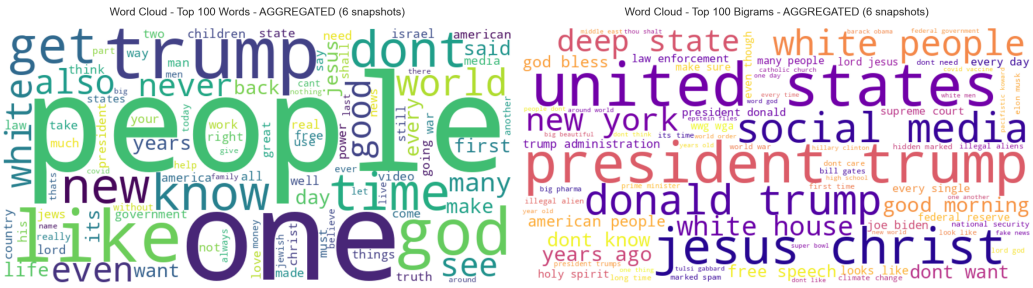

Figure 2.7: Word cloud of the aggregated dataset. Prominent terms include political figures, common verbs, and thematic keywords reflecting the platform's discourse patterns.

Figure 2.8 shows the top words and bigrams, providing insight into the most common single terms and two-word phrases.

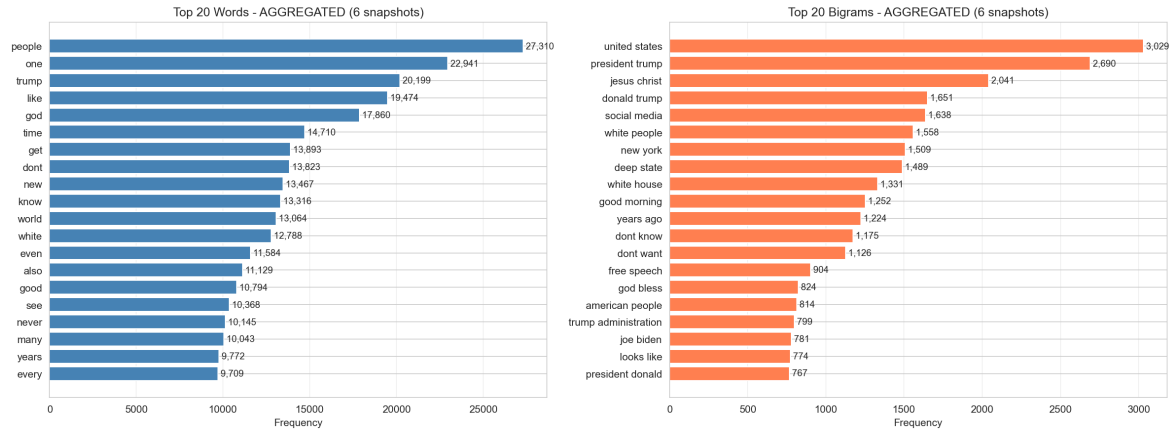

Figure 2.8: Top 20 words and bigrams (aggregated). The left panel shows the most frequent single words, while the right panel displays the most common two-word phrases, revealing common discourse patterns.

2.4 Verify Data Quality

Data quality verification examines completeness, consistency, accuracy, and potential issues in the dataset.

Completeness Analysis

Missing Values

The dataset shows excellent completeness for most attributes. The only attribute with missing values is language, with 877 missing values (0.48%) in the aggregated current snapshots and 3,747 (0.85%) in the incremental data. All other attributes are 100% complete.

Table 2.6: Missing values summary (aggregated current snapshots)

Attribute	Missing Count	Missing %
language	877	0.48%
All other attributes	0	0.00%

The missing language values likely correspond to posts where automatic language detection failed, possibly due to very short content, mixed-language posts, or non-standard text.

No Empty Content

Importantly, no posts with empty content fields were found, ensuring all records contain textual data for analysis.

Consistency Checks

Snapshot vs. Incremental Consistency

A critical quality check verified that each current snapshot is fully contained within its corresponding incremental file. All six snapshots achieved 100% coverage:

Table 2.7: Snapshot-incremental consistency verification

Snapshot	Snapshot Posts	Incremental Posts	Coverage
2016-2021	18,926	18,926	100%
2022	13,108	32,034	100%
2023	13,376	45,410	100%
2024	13,400	58,810	100%
Jan-Jul 25	44,878	103,688	100%
Jul 25	78,857	182,545	100%

Temporal Consistency

The created_at and timestamp fields are identical across all snapshots, confirming consistent timestamp handling. No posts with future dates were detected.

Unique Identifiers

No duplicate post IDs were found, either within individual snapshots or across the aggregated dataset. Each post maintains a unique identifier throughout the dataset.

Duplicate Content Analysis

While post IDs are unique, content-level duplicates exist in the dataset. Table 2.8 summarizes the duplicate content findings.

Table 2.8: Duplicate content analysis per snapshot

Snapshot	Duplicate Posts	Percentage
2016-2021	339	1.79%
2022	396	3.02%
2023	563	4.21%
2024	563	4.20%
Jan-Jul 25	3,377	7.52%
Jul 25	7,919	10.04%

The increasing duplicate rate over time suggests growing repetitive posting behavior, possibly including:

- Users reposting the same content multiple times;
- Common phrases or memes shared by different users;
- Automated or bot-like posting patterns.

Importantly, no completely duplicate rows (excluding ID) were found, meaning each post has unique metadata even when content is identical.

Data Type Verification

All attributes maintain consistent data types across snapshots:

- Numeric fields (id, account_id, replies_count, reblogs_count, quotes_count): int64

- Text fields (content, language, created_at, timestamp, reactions_counts): str

Encoding Issues

No encoding issues were detected in the text content. The dataset appears to be properly UTF-8 encoded, with no problematic character sequences found during quality checks.

Outlier Detection

Several extreme values were identified but are considered legitimate data points:

- Maximum replies: 22,211 (highly viral post)
- Maximum reblogs: 24,464 (highly viral post)
- Maximum reactions: 116,141 (exceptionally popular content)
- Maximum text length: 11,753 characters (long-form content)

These outliers represent the heavy-tailed distribution typical of social media engagement, where a small number of viral posts receive disproportionate attention.

Summary of Data Quality

The dataset demonstrates high overall quality:

- **Completeness**: 99.52% complete (only language has missing values)
- **Consistency**: 100% snapshot-incremental consistency; unique IDs throughout
- **Accuracy**: No encoding issues; timestamps properly formatted
- **Timeliness**: Data spans 2016-2025 with recent snapshots being most comprehensive

Areas requiring attention during data preparation include:

1. Handling the 0.48% missing language values (imputation or filtering)
2. Addressing content duplicates based on analysis objectives
3. Considering the heavy-tailed engagement distributions for modeling purposes

2.5 Summary

The data understanding phase revealed a rich dataset of 182,545 posts from the Gab social network, spanning nearly a decade (2016-2025). Key findings include:

- **Growth pattern**: The platform shows exponential growth, with 2025 data alone representing 68% of all posts
- **Language homogeneity**: 97% English content, making it suitable for single-language NLP tasks
- **Content characteristics**: Short-form content dominates (median 89 characters, 13 words) with heavy-tailed distributions
- **Engagement patterns**: Highly skewed distributions with 28% zero-engagement posts and viral outliers reaching 20,000+ interactions

- **User dynamics**: Power-law distribution with 0.43% of users (116 power users) driving significant content volume
- **Data quality**: High completeness (99.52%) with only the language field showing missing values; perfect snapshot-incremental consistency
- **Duplicate trend**: Increasing content duplication over time (from 1.79% to 10.04%), suggesting potential bot activity or viral content resharing

The dataset is well-suited for link prediction and social network analysis tasks, with the temporal snapshots enabling longitudinal studies of network evolution. The preprocessing pipeline should address the minor missing values and consider strategies for handling the heavy-tailed engagement distributions.

3 Data Preparation

This chapter covers the data preparation phase of the CRISP-DM methodology.

3.1 Select Data

Select the relevant data for the analysis.

3.2 Clean Data

Clean the data by handling missing values, outliers, and errors.

3.3 Construct Data

Construct new features or transform existing ones.

3.4 Integrate Data

Integrate data from multiple sources.

3.5 Format Data

Format the data according to the requirements of the modeling tools.

4 Modeling

This chapter covers the modeling phase of the CRISP-DM methodology.

4.1 Select Modeling Techniques

For dynamic graph analysis, we selected **EvolveGCN** (Evolving Graph Convolutional Networks) [8] as our primary modeling approach. This technique addresses a critical limitation of existing methods: the requirement for nodes to be present throughout the entire time span (training and testing), making them less applicable when node sets frequently change or completely differ between time steps.

Motivation and Context

Traditional graph neural network approaches for dynamic graphs typically resort to node embeddings and use recurrent neural networks to regulate these embeddings and learn temporal dynamics. However, this approach faces significant challenges in practical scenarios where:

- New nodes emerge after training
- Nodes frequently appear and disappear
- Node sets at different time steps may completely differ

EvolveGCN resolves these challenges by focusing on model adaptation rather than node embeddings. Instead of using RNNs to learn dynamics from node features extracted by GNNs, EvolveGCN uses RNNs to regulate the GCN model (network parameters) at every time step [8]. This approach effectively performs model adaptation without requiring nodes to be present throughout the entire time span, making it suitable for our social network analysis where users may join or leave the platform.

Architecture Overview

EvolveGCN adapts the Graph Convolutional Network (GCN) model along the temporal dimension without resorting to node embeddings. The proposed approach captures the dynamism of the graph sequence through using an RNN to evolve the GCN parameters, with two architectures considered for parameter evolution [8].

At each time step t, the input data consists of the pair $(A_t \in \mathbb{R}^{n \times n}, X_t \in \mathbb{R}^{n \times d})$, where A_t is the graph adjacency matrix and X_t is the matrix of node features (each row is a d-dimensional feature vector).

Graph Convolution Layer

A GCN consists of multiple layers of graph convolution, which is similar to a perceptron but additionally has a neighborhood aggregation step motivated by spectral convolution. At time t, the l-th layer takes the adjacency matrix A_t and the node embedding matrix $H_t^{(l)}$ as input, and uses a weight matrix $W_t^{(l)}$ to update the node embedding matrix to $H_t^{(l+1)}$ as output:

$$H_t^{(l+1)} = \text{GCONV}(A_t, H_t^{(l)}, W_t^{(l)}) := \sigma(\hat{A}_t H_t^{(l)} W_t^{(l)}) \tag{4.1}$$

where $\hat{A}_t$ is a normalization of A_t defined as:

$$\hat{A} = \tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}}, \quad \tilde{A} = A + I, \quad \tilde{D} = \text{diag}\left(\sum_j \tilde{A}_{ij}\right) \tag{4.2}$$

and σ is the activation function (typically ReLU) for all but the output layer. The initial embedding matrix comes from the node features: $H_t^{(0)} = X_t$. For a GCN with L layers, the output layer function σ may be the identity (producing high-level node representations) or softmax (for node classification probabilities).

Weight Evolution Mechanisms

At the heart of the proposed method is the update of the weight matrix $W_t^{(l)}$ at time t based on current as well as historical information. This requirement can be naturally fulfilled by using a recurrent architecture, with two options.

EvolveGCN-H (Hidden State Version): The first option treats $W_t^{(l)}$ as the hidden state of the dynamical system. A Gated Recurrent Unit (GRU) is used to update the hidden state upon time-t input to the system, where the input information is naturally the node embeddings $H_t^{(l)}$:

$$W_t^{(l)} = \text{GRU}(H_t^{(l)}, W_{t-1}^{(l)}) \tag{4.3}$$

The implementation requires two extensions from standard GRU: (a) extending inputs and hidden states from vectors to matrices (because the hidden state is now the GCN weight matrices), and (b) matching the column dimension of the input with that of the hidden state. This is achieved through a summarization function that condenses the node embedding matrix $H_t^{(l)}$ into a matrix with appropriate dimensions using a learned parameter vector to compute weights for the rows, selecting the top-k weighted rows.

EvolveGCN-O (Output Version): The second option treats $W_t^{(l)}$ as the output of the dynamical system (which becomes the input at the subsequent time step). A Long Short-Term Memory (LSTM) cell models this input-output relationship, maintaining system information through a cell context that acts like the hidden state of a GRU:

$$W_t^{(l)} = \text{LSTM}(W_{t-1}^{(l)}) \tag{4.4}$$

In this version, node embeddings are not used at all. Implementing the -O version requires only a straightforward extension of the standard LSTM from the vector version to the matrix version.

Evolving Graph Convolution Unit (EGCU)

Combining the graph convolution unit GCONV and a recurrent architecture, we reach the Evolving Graph Convolution Unit (EGCU). Depending on the way GCN weights are evolved, we have two versions:

For EvolveGCN-H:

$$W_t^{(l)} = \mathrm{GRU}(H_t^{(l)}, W_{t-1}^{(l)}) \tag{4.5}$$
$$H_t^{(l+1)} = \mathrm{GCONV}(A_t, H_t^{(l)}, W_t^{(l)}) \tag{4.6}$$

For EvolveGCN-O:

$$W_t^{(l)} = \mathrm{LSTM}(W_{t-1}^{(l)}) \tag{4.7}$$
$$H_t^{(l+1)} = \mathrm{GCONV}(A_t, H_t^{(l)}, W_t^{(l)}) \tag{4.8}$$

In the -H version, the GCN weights are treated as hidden states of the recurrent architecture; in the -O version, these weights are treated as input/outputs. Chaining the units bottom-up creates a GCN with multiple layers for one time step, and unrolling over time horizontally forms a lattice on which information ($H_t^{(l)}$ and $W_t^{(l)}$) flows.

4.2 Generate Test Design

Metrics

- **Classical:** Precision, Recall, F1-Score, AUC-ROC, MAP.
- **Graph-based:** Average Degree, Clustering Coefficient, Modularity, Community structure, GCC ratio.

We compute these metrics both on the final graph obtained from the snapshots and on the graph restricted to synthetic nodes (i.e., the graph used to complete the synthetic dataset).

Experiments

Planned experiments: degree-based features; embedding-based features; embedding features in a "live-update" setting (if feasible). Model variants: both the H and O versions, when possible.

Examples of network metrics

Facebook Social Circles Average Degree: 43.6910; Average Shortest Path: 3.6925; Modularity: 0.8348.

Twitter Social Circles Average Degree: 21.75; Modularity: 0.8042; Average Shortest Path: 4.9058. Nodes: 81 306; Edges: 1 768 149.

Google+ Social Circles Average Degree: 127.06; Modularity: 0.4742; Average Shortest Path: 3.3163. Nodes: 107 614; Edges: 13 673 453.

Description of metrics

We analyze graphs using three fundamental metrics:

- **Average Degree:** the mean number of connections per node. For undirected graphs

$$\mathrm{AvgDeg} = \frac{2|E|}{|V|},$$

  while for directed graphs we use

$$\mathrm{AvgDeg} = \frac{|E|}{|V|},$$

where $|E|$ and $|V|$ denote the number of edges and vertices, respectively.

- **Average Shortest Path:** the average number of hops between all pairs of reachable nodes within the largest connected component (LCC). For directed graphs we consider weakly connected components.
- **Modularity:** a measure of community structure quality, typically ranging from -1 to 1. High positive values (e.g., > 0.3) indicate strong community structure. We compute modularity using the Louvain algorithm; for directed graphs we first convert to an undirected representation.

4.3 Build Model

In this project, we explored two distinct training pipelines using the H variant of the EvolveGCN architecture.

Standard Implementation

The first methodology closely follows the reference implementation of EvolveGCN:

- **Data Splitting**: Employs a standard proportion-based split or Leave-One-Out Cross-Validation (LOOCV). The timeline is divided into fixed blocks: typically the first 70% for training, 10% for validation, and 20% for testing.
- **Sequence-based Training**: The model processes the entire training sequence in a single batch or over multiple epochs. It learns the "average" dynamics of the training set but remains static once the training phase is complete.
- **Memory Management**: Uses a fixed global node index system. While simpler to implement, it can become inefficient for graphs with a large total number of nodes where only a small subset is active at any given time step.
- **Initialization**: Adheres to *Kaiming* initialization for weight matrices.

Incremental Implementation

The second methodology is an adapted version designed specifically for *incremental learning* considering the cumulative assumption of our dataset.

Training Strategy: Snapshot-by-Snapshot Adaptation

Unlike the static approach, the incremental implementation uses a **Phase-based Training** workflow. The timeline is discretized into multiple temporal snapshots $\{S_0, S_1, \dots, S_n\}$. In each training phase i:

1. The model is initialized with the weights from the previous phase $i - 1$.
2. It is fine-tuned or trained on the current snapshot S_i (representing "today's" graph).
3. It is immediately evaluated on its ability to predict links in the subsequent snapshot S_{i+1} (representing "tomorrow's" graph).

So, instead of passing to the model all the training snapshot data at once, we train it incrementally on each snapshot, allowing it to adapt its weights on the single snapshot and then immediately test its performance on the next snapshot. This simulates a real-world scenario where the model must adapt to new data as it arrives, rather than relying on a static training set.

Walk-Forward (Rolling) Evaluation

To accurately measure performance in a dynamic environment, we implement **Walk-Forward Validation**. This mimics a real-world deployment scenario:

- **Future Prediction**: For every snapshot t, the model's performance is reported based on its prediction accuracy for snapshot $t + 1$.
- **Evaluation Metrics**: We track traditional link prediction metrics (AUC-ROC, Mean Average Precision) alongside structural graph metrics (Modularity, GCC, Community Detection) for each predicted graph window.

4.4 Assess Model

5 🔢 Evaluation

This chapter covers the evaluation phase of the CRISP-DM methodology.

5.1 Evaluate Results

5.2 Review Process

6 Deployment

Up to now, no deployment of the model has been planned. The model is still in evolution and further improvements are needed to make it suitable for real-world deployment.

7 🚀 Conclusion

Bibliography

[1] Shazia Tabassum, Fabiola S. F. Pereira, Sofia Fernandes, and Jo
ao Gama. Social network analysis: An overview. *WIREs Data Mining and Knowledge Discovery*, 8(5):e1256, September 2018. 1, 2

[2] Nur Nasuha Daud, Siti Hafizah Ab Hamid, Muntadher Saadoon, Firdaus Sahran, and Nor Badrul Anuar. Applications of link prediction in social networks: A review. *Journal of Network and Computer Applications*, 166:102716, September 2020. 1, 2

[3] Antonio Pellicani, Gianvito Pio, Domenico Redavid, and Michelangelo Ceci. SAIRUS: Spatially-aware identification of risky users in social networks. *Information Fusion*, 92:435–449, April 2023. 1

[4] Zafran Khan, Zeeshan Khan, Byung-Geun Lee, Hong Kook Kim, and Moongu Jeon. Graph neural networks based framework to analyze social media platforms for malicious user detection. *Applied Soft Computing*, 155:111416, April 2024. 1

[5] Francesco Benedetti, Antonio Pellicani, Gianvito Pio, and Michelangelo Ceci. Multi-view Learning for the Identification of Risky Users in Dynamic Social Networks, March 2025. 1

[6] Francesco Benedetti, Antonio Pellicani, Gianvito Pio, and Michelangelo Ceci. IMMENSE: Inductive Multi-perspective User Classification in Social Networks. *Online Social Networks and Media*, 50:100335, December 2025. 1, 2

[7] Roberto Corizzo, Gianvito Pio, Emanuele Pio Barracchia, Antonio Pellicani, Nathalie Japkowicz, and Michelangelo Ceci. HURI: Hybrid user risk identification in social networks. *World Wide Web*, 26(5):3409–3439, September 2023. 1

[8] Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, Tao Schardl, and Charles Leiserson. EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs. 34(04):5363–5370. 16